

Java 8 Programs

Intermediate Java Stream Questions (40 tasks)

Grouping & Collecting

1. Group a list of objects (e.g., `Employee`) by a field (e.g., department).
2. Count elements in each group using `Collectors.counting()`.
3. Find the average value in each group (e.g., average salary by dept).
4. Create nested grouping (group by department, then city).
5. Convert a grouped `Map` into another format (e.g., list of entries).
6. Group and then sort elements within each group.
7. Group and compute max/min within groups.
8. Group and collect into sets instead of lists.
9. Group by predicate (like even/odd numbers).
10. Group a stream of strings by length.

Skip / Limit / Pagination

11. Return top N elements after sorting.
12. Skip the first N elements and collect the rest.
13. Pagination — skip + limit together.
14. Skip elements with a predicate (via filter + skip).
15. Use `limit` to take only the first N unique values.
16. Find first N strings above a certain length.

- 17. From a large list, return a sublist after skipping some elements.
- 18. Combine skip + limit to simulate page k of results.

Second-Highest / Top-N Logic

- 19. Find the second-largest number in a list.
- 20. Find the second-smallest number in a list.
- 21. Find the second-highest salary in a list of employees.
- 22. Find the n -th largest value using skip + sorted.
- 23. Return the top 3 elements sorted in reverse order.
- 24. Get the top N elements per group (e.g., highest salaries per department).
- 25. From a stream of grades, find the second highest unique score.

Partitioning & Filtering

- 26. Partition a list into two categories based on a predicate.
- 27. Filter elements with multiple conditions using Streams.
- 28. Filter out nulls and empty strings.
- 29. Use `anyMatch`, `allMatch`, `noneMatch` to check conditions.
- 30. Use `peek()` to inspect values in a long pipeline.

Mapping & Transformations

- 31. Flatten a list of lists using `flatMap()`.
- 32. Transform a collection of objects to another type.
- 33. Count characters in strings and group by character.

- 34. Convert a String into a map of character → count.
- 35. Build a map from two fields (e.g., id → name).

Collectors & Custom Logic

- 36. Collect elements into a `TreeSet` for sorted unique output.
- 37. Custom collector to join strings with a delimiter.
- 38. Use `Collectors.toMap()` with merge functions to handle duplicates.
- 39. Create a map of boolean keys (true/false) to element lists.
- 40. Build a `Map<Dept, Double>` of sum of values instead of lists.